48

BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

ScopedValue Devrimi: Java 25 ile Değişmez, Miras Alınabilir & Sıfır Bellek Bağlamı

ScopedValue in Java 25: Immutable, Inheritable, Zero-Allocation Context

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────────────┐
│                    SCOPEDVALUE VS THREADLOCAL                        │
└──────────────────────────────────────────────────────────────────────┘

Özellik                 | ThreadLocal                    | ScopedValue (Java 25)
------------------------|--------------------------------|-----------------------------
Değiştirilebilirlik     | Mutable (Herkes değiştirebilir)| IMMUTABLE (Kesinlikle Değişmez!)
Yaşam Döngüsü           | Belirsiz (remove() unutulur)   | Sınırlı (Scope kapanınca yok olur)
Bellek Maliyeti         | Her thread için ayrı kopya!    | SIFIR KOPYA (Paylaşımlı Referans)
Sanal Thread Uyumu      | Kötü (OOM Riski)               | MÜKEMMEL (Milyonlarca VT destekler)

Kullanım Modeli:
ScopedValue.where(TENANT_ID, "ACME_CORP")
           .run(() -> {
               // Sadece bu blok ve altındaki sanal thread'ler okuyabilir!
               service.handleRequest();
           }); // Blok bittiğinde bağlam otomatik temizlenir!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

ThreadLocal, şirkette her personelin cebine koyduğu ve canı isteyenin üzerine kurşun kalemle yazı yazabildiği bir kâğıttır. Biri kâğıdı silmeyi unutursa sonraki vardiyadaki kişi yanlış bilgiyle çalışır. ScopedValue ise toplantı odasının kapısına asılan mühürlü bir levhadır: 'Bu odadayken şirket ACME Corp'tur'. İçerideki herkes levhayı okuyabilir ama kimse değiştiremez. Odadan çıktığınız anda levha geçerliliğini yitirir; kimsenin cebinde çöp kalmaz!

🇬🇧 ENGLISH (ELI5)

`ThreadLocal` is handing every employee an erasable notepad that anyone can scribble on at any time. If someone forgets to erase it, the next shift inherits corrupted state. `ScopedValue` is an immutable badge worn strictly during a specific meeting. Everyone inside the room can inspect the badge, no one can tamper with it, and the moment you step outside the door, the badge dissolves into thin air. Zero leaks, zero copies!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ScopedValue Mimarisi ve JVM Optimizasyonları: 1) Değişmezlik (Immutability): `ScopedValue\<T>` değerleri bağlandıktan sonra (`where`) asla değiştirilemez (`set` metodu yoktur). Yalnızca içiçe yeni bir kapsam (`rebinding`) açılabilir. 2) Dinamik Kapsam (Dynamic Scoping): Değer sadece `run` veya `call` bloğu yürütülürken mevcuttur; blok bittiğinde GC otomatik olarak erişim bağlarını serbest bırakır; manuel `remove()` çağırma gerekliliği ve bellek sızıntısı riski ortadan kalkar. 3) Sıfır Tahsisli Miras (Zero-Allocation Inheritance): Alt sanal thread'ler (`StructuredTaskScope`) çatallandığında ana thread'in ScopedValue değerlerini kopyalamaz; aynı değişmez referansı doğrudan paylaşırlar. Bu da milyonlarca sanal thread açıldığında bellek tüketimini sıfıra indirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot ScopedValue Architecture & Mechanical Sympathy: 1) Strict Immutability: Unlike `ThreadLocal`, `ScopedValue\<T>` exposes no `.set()` method. Once bound via `ScopedValue.where()`, the value is read-only for the duration of the callable invocation. Nested scopes may dynamically shadow (rebind) values without mutating parent states. 2) Bounded Extent: The context lifetime strictly mirrors the execution lifetime of the enclosing lambda, guaranteeing deterministic cleanup without relying on defensive `.remove()` invocations. 3) Zero-Copy Fork Inheritance: When child virtual threads are spawned via `StructuredTaskScope`, they inherit access to enclosing scoped values without duplicating map entries or allocating heap buffers. HotSpot optimizes ScopedValue lookups using JIT-compiled intrinsic cache lines.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ScopedValue ile Güvenli ve Değişmez Güvenlik Bağlamı Taşıma
import java.util.concurrent.StructuredTaskScope;

public class ScopedValueModernDemo {
    // Statik ve değişmez ScopedValue tanımları
    public static final ScopedValue<String> TENANT_ID = ScopedValue.newInstance();
    public static final ScopedValue<String> USER_ROLE = ScopedValue.newInstance();

    static void executeDatabaseQuery() {
        // İhtiyaç duyulan yerde bağlamı anında ve güvenle oku!
        System.out.printf("[DB] Tenant: %s | Role: %s | Thread: %s%n",
            TENANT_ID.get(), USER_ROLE.get(), Thread.currentThread());
    }

    void main() throws Exception {
        // Bağlamı sınırlı bir kapsamda (scope) bağla ve çalıştır
        ScopedValue.where(TENANT_ID, "FINTECH_CORP_01")
                   .where(USER_ROLE, "ADMIN")
                   .run(() -> {
                       System.out.println("Kapsam içine girildi...");
                       executeDatabaseQuery();

                       // Alt sanal thread'ler de bu bağlamı sıfır maliyetle paylaşır!
                       try (var scope = new StructuredTaskScope<Void>()) {
                           scope.fork(() -> {
                               executeDatabaseQuery();
                               return null;
                           });
                           scope.join();
                       } catch (InterruptedException _) {}
                   });

        // Kapsam dışına çıkıldığında değerler otomatik olarak erişilemez olur!
        System.out.println("Kapsam bitti. TENANT_ID bağlı mı? " + TENANT_ID.isBound());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

ScopedValue.get() metodunu where() ile bağlanmamış bir kapsamda çağırırsanız NoSuchElementException fırlatılır! Güvenli kontrol için isBound() veya orElse() metotlarını kullanabilirsiniz.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Invoking `ScopedValue.get()` outside of an active `.where()` binding scope throws an immediate `NoSuchElementException`. Always guard optional scopes using `.isBound()` or provide a default via `.orElse(defaultValue)`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• ScopedValue değişmezdir, kapsamı belirlidir ve miras alındığında sıfır bellek tüketir. ThreadLocal'ın modern ve güvenli alternatifidir.

🎯 Key Takeaways for the Interview

• ScopedValue is immutable, lexically bounded, and zero-allocation upon thread inheritance, replacing hazardous ThreadLocal in modern Java.